SE LAB-05

Lab 5: Static Code Analysis

Name: Narendra Kathare

SRN: PES2UG25CS814

SEC: J

Github Link Before Changes (Assigned Problem)

https://github.com/SETAPESU26/L5_61_attendance-tracker

Github Link After Changes

https://github.com/narendrakathare03/L5_61_attendance-tracker

Issue Documenta on Table (issue.md)

Issue Documentation Table

Issue	Type	Original Location	Description	Fix Approach
Hardcoded ERP API key	Security	Global configuration	API key was stored directly in source code.	Read the key from the `ERP_API_KEY` environment variable.
Disabled TLS verification	Security	`syncToErp()`	`verify=False` disabled HTTPS certificate verification.	Use default certificate verification.
Shell command execution	Security	`backupReports()`	`subprocess.Popen()` used `shell=True`.	Replace shell execution with Python's `tarfile` module.
Unsafe reminder command	Security	`scheduleReminders()`	Command string was configured for shell execution.	Use a list of arguments with `shell=False`.
Mutable default argument	Bug	`markAttendance()`	`register={}` could share state across calls.	Use `None` and initialize a new dictionary.
Incorrect eligibility boundary	Logic Bug	`isEligible()`	Exactly 75% attendance was rejected.	Change `>` to `>=`.

Division by zero	Reliability	`attendancePercentage()`	Empty records could cause division by zero.	Return `0.0` when no records exist.
Unsafe CSV handling	Reliability	`exportCsv()`	File handling lacked a context manager and explicit encoding.	Use `with open(...)` and UTF-8 encoding.
Spreadsheet formula injection	Security	`main()` / `exportCsv()`	Formula-like student names could be interpreted as spreadsheet formulas.	Sanitize potentially dangerous CSV cell values.
Deprecated TLS protocol	Security	`legacyChannel()`	TLS 1.0 was used as the default protocol.	Remove deprecated TLS usage.
Poor naming and formatting	Code Quality	Multiple functions	CamelCase names and inconsistent formatting reduced readability.	Use snake_case and consistent formatting.
Empty attendance history	Reliability	`latestDate()`	Finding the maximum date could fail for empty records.	Return `None` when there are no records.

Answers – Reflection on Questions - Reflection.md

Lab 5: Static Code Analysis — Reflection

1. What is static code analysis?

Static code analysis is the process of examining source code without executing the program. It helps developers identify potential programming errors, security vulnerabilities, and style violations before deployment.

2. What is the difference between Pylint, Bandit, and Flake8?

Pylint checks Python code quality, naming conventions, and possible programming errors. Bandit identifies common security vulnerabilities and unsafe coding patterns. Flake8 checks formatting, whitespace, and compliance with Python style conventions, including many PEP 8 rules.

3. What issues did you identify and fix during this lab?

I addressed multiple issues in the attendance tracker. These included a hardcoded ERP API key, disabled TLS certificate verification, shell-based command execution, a mutable default argument, and an incorrect attendance eligibility condition. I also improved empty-record handling, CSV file management, spreadsheet formula injection protection, and code formatting.

These changes helped make the program more secure, reliable, and maintainable.

4. How does static code analysis improve software quality?

Static code analysis helps detect potential defects early in development. It improves security, encourages consistent coding practices, makes code easier to read, and reduces maintenance effort. However, static analysis should be combined with program execution and functional testing because it cannot detect every possible runtime or logical problem.